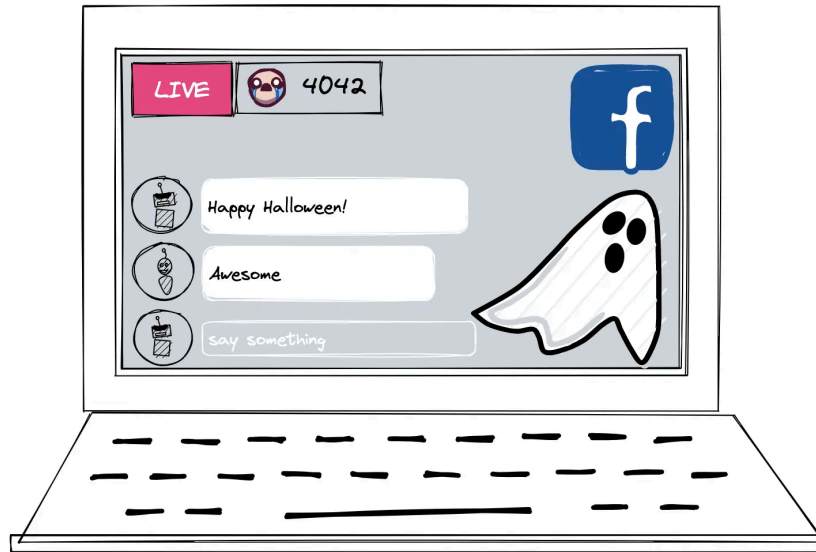


Live Comment System Design

Real-Time Live Commenting Platform

👤 Neo Kim included in [Deep Dive](#)

📅 2023-04-08 ✍️ 9373 words ⌚ 45 minutes



The target audience for this article falls into the following roles:

- Tech workers
- Students
- Engineering managers

The prerequisite to reading this article is fundamental knowledge of system design components. This article does not cover an in-depth guide on individual system design components.

Disclaimer: The system design questions are subjective. This article is written based on the research I have done on the topic and might differ from real-world implementations. Feel free to share your feedback and ask questions in the comments.



Download my system design playbook for free on newsletter signup:



[The System Design Newsletter](#)

Download my system design playbook on newsletter signup for FREE

By Neo Kim 🏆 · Over 235,000 subscribers

amanpunetha08@gmail.com



By subscribing you agree to [Substack's Terms of Use](#), our [Privacy Policy](#) and our [Information collection notice](#)

substack

How Does the Live Comment Work?

At a **high level**, the Live Comment service performs the following operations:

- gateway server fans out live comments to the clients through [server-sent events \(SSE\)](#)

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

- in-memory subscription store on the gateway server keeps the viewership associativity
- gateway server subscribes to a live video on the endpoint store
- endpoint store keeps the set of gateway servers subscribed to a particular live video
- heartbeat signal or time to live (**TTL**) on keys on the subscription store can be used to drain the inactive SSE connections
- ending of a live video can trigger an update on the endpoint store
- dispatcher broadcasts the live comments to the dispatcher in peer data centers

What is a Live Video?

Live video is the streaming of video over the internet in real-time without prior recording and storage. Television broadcasts, video game streams, and social media videos are often live-streamed ¹.



Figure 1: Facebook Likes; Typing indicators; Live comments

Live video is different from regular videos because live video results in spiky traffic patterns. Usually, Live videos are more engaging and tend to be watched thrice more than regular videos. On top of that, social networks such as Facebook show live videos at the top of the newsfeed, thereby increasing the probability of live videos being watched by the users (**clients**) ².

What are Live Comments?



00:04

Live comments; Source: video by [Joseph Redfield Nino](#) from [Pixabay](#)

Real-time experience makes the platform feel like a place of activity for the clients ³. Live commenting is a feature that allows clients to publish real-time comments on live videos. The live comments are usually a mixture of feedback from the clients on the live video or casual conversations between clients. Besides, live comments enable the streamer of the live video to engage with the clients during the live video broadcast.

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

The following terminology might be helpful for you:

- Node: a server that provides functionality to other services
- Data replication: a technique of storing multiple copies of the same data on different nodes to improve the availability and durability of the system
- Data partitioning: a technique of distributing data across multiple nodes to improve the performance and [scalability](#) of the system
- API: a software intermediary that allows two services to talk with each other
- Fault tolerance: the ability of a service to recover from a failure without losing data
- High availability: the ability of a service to remain reachable and not lose data even when a failure occurs

Questions to ask the Interviewer

| Candidate

1. What are the primary use cases of the system?
2. Do live commenting service support only text?
3. Are the clients distributed across the globe?
4. What is the amount of Daily Active Users (**DAU**)?
5. What is the system's total number of daily live videos?
6. What is the average amount of live comments on a live video?
7. What is the anticipated read: write ratio of live comments?
8. What is the peak amount of concurrent users watching the same live video?
9. Should the comments on streamed videos be archived to save storage?

| Interviewer

1. Clients can interact with each other in real-time over live comments on a Facebook live video
2. Yes
3. Yes
4. 100 million DAU
5. 200 million daily live videos
6. 10
7. 100: 1
8. 80 million
9. Yes, there is no need to support the replay of live comments

Requirements

| Functional Requirements

- The **publishers** (client) can write real-time live comments on a Facebook live video
- The **receivers** (clients) watching the Facebook live video should be able to view the live comments in real time
- The receiver should be able to view an active real-time feed of live comments on each live video while scrolling across the Facebook newsfeed
- The **typing indicators** (somebody is typing feature) should be supported
- The total count of comments should be visible on each live video
- The live comment service should handle clients across the globe
- The live comment service should handle millions of concurrent clients

| Non-Functional Requirements

- Highly availability
- Fault-tolerant

- Eventual [consistency](#)

Live Commenting API Design

The real-time Application Programming Interface (**API**) can be implemented for faster user experiences and instant delivery of live comments. The average time for a human to blink is 100 ms, and the average reaction time for a human is around 250 ms. Therefore, the actions performed within 250 ms are perceived as real-time or live ^{4, 5}. An event-driven architecture can be used to build a real-time data platform. The general subscription models for an API are the following ⁶:

- push-based (server-initiated)
- pull-based (client-initiated)

The popular protocols for an event-driven API are the following ⁶:

Protocol	Description	Use Cases	Subscription Model
Webhook	HTTP-based callback function that allows lightweight, event-driven infrequent communication between APIs	trigger automation workflows	push-based
WebSub	communication channel for frequent messages between web content publishers and subscribers based on HTTP webhooks	news aggregator platforms, stock exchanges, and air traffic networks	push-based
WebSockets	provides full-duplex communication channels over a single TCP connection with lower overhead than half-duplex alternatives such as HTTP polling	financial tickers, location-based apps, and chat solutions	pull-based
SSE	lightweight and subscribe-only protocol for event-driven data streams	live score updates	pull-based
MQTT	protocol for streaming data between devices with limited CPU power and low bandwidth networks	Internet of Things	pull-based

The communication protocol and subscription model chosen should be ideal to build a reliable and scalable live comment service. The pull-based subscription model is the optimal choice for the delivery of live comments to the receivers for the following reasons ⁶:

- the receiver only needs to view the live comments when the receiver is online (connected or subscribed to the live video)
- the receiver can ignore the comments when disconnected or unsubscribed
- non-trivial to predict the total count of clients that will be subscribed to watching a live video
- the receivers will be geographically distributed across the globe
- the connection from the receiver might be unpredictable



Figure 2: REST compared with SSE

The client creates a regular HTTP long poll connection with the server with [server-sent events \(SSE\)](#). The server can push a continuous stream of data to the client on the same connection as events occur. The client doesn't need to perform subsequent requests ⁷.



Figure 3: HTTP long poll with SSE

The only difference for SSE from a regular HTTP request is that the *Accept* header on the HTTP request holds the value *text/event-stream*. The [EventSource interface](#) is used by the client to receive and process server-sent events independently in *text/event-stream* format without closing the connection. All modern web browsers support the EventSource interface natively. The EventSource interface can be implemented on [iOS](#) and [Android](#) platforms with lightweight libraries ^{7, 8}.



Figure 4: Complexity inversion in an event-driven API

The client is responsible for maintaining the state and execution of requests in the Representational state transfer (**REST**) paradigm. On the contrary, the server is responsible for maintaining the state and pushing updates to the client in an event-driven architecture. In layman's terms, the complexity of the receiver is kept minimum in an event-driven API as the receiver only has to create an SSE connection to view the live comments on a Facebook live video ⁶. In conclusion, SSE is the optimal protocol for the live comment service due to the following reasons ^{7, 8}:

- SSE works over traditional HTTP
- SSE streams chunks of data over the same open HTTP connection

The predefined fields of an SSE connection are the following ⁶:

Field	Description
event	the event type defined by the server
data	the payload of the event
id	ID for each event
retry	the client attempts to reconnect with the server after a specific timeframe if the connection was closed

The following are the drawbacks of SSE ⁶:

- the data format is restricted to transporting UTF-8 messages with no support for binary data
- only up to six concurrent SSE connections can be opened per web browser on pre-HTTP/2 networks

The components in the system expose the Application Programming Interface (API) endpoints to the client through Representational State Transfer (REST). The description of HTTP **Request headers** is the following:

Header	Description
accept	type of content the client can understand
authorization	authorize your user account
content-encoding	compression type used by the data payload
method	HTTP Verb

content-type	type of data format (JSON or XML)
--------------	-----------------------------------

The description of HTTP **Response headers** is the following:

Header	Description
status code	shows if the request was successful
content-type	type of data format

| How does the receiver subscribe to a specific live video?

The client must subscribe to a live video for viewing the live comments. The client executes an HTTP PUT request for subscribing to a live video. The PUT requests are idempotent. The PUT method is used instead of the GET method because the in-memory subscription store will be modified when a client subscribes to a live video.

▼ JavaScript 

```
1 /videos/:video-id/subscriptions
2 method: PUT
3 accept: text/event-stream
4 authorization: Bearer <JWT>
5 content-length: 20
6 content-type: application/json
7 content-encoding: gzip
8
9 {
10   user_id: <int>
11 }
```

The *accept: text/event-stream* HTTP request header indicates that the client is waiting for an open connection to the event stream from the server to fetch live comments⁹. The server responds with status code *200 OK* on success.

▼ JavaScript 

```
1 status code: 200 OK
2 content-type: text/event-stream
```

The *content-type: text/event-stream* HTTP response header indicates that the server established an open connection to the event stream to dispatch events to the client. The response event stream contains the live comments.

▼ JavaScript 

```
1 id: 1
2 event: comment
3 data: {"awesome"}
4 data: {"hey there"}
```

| How does the receiver unsubscribe from a live video?

The client should unsubscribe from a live video to stop receiving live comments. The client executes an HTTP DELETE request for unsubscribing from a live video⁹. The DELETE requests are idempotent.

▼ JavaScript 

```
1 /videos/:video-id/subscriptions/:subscription-id
2 method: DELETE
3 authorization: Bearer <JWT>
```

The server responds with status code *200 OK* on success.

▼ JavaScript 

```
1 status code: 200 OK
```

The server responds with status code *204 No Content* when the client successfully unsubscribes from a live video. The HTTP response body will be empty.

▼ JavaScript 

| How does the client publish a live comment?

The client executes an HTTP POST request to publish a live comment.

▼ JavaScript



```
1 /videos/:video-id/comments
2 method: POST
3 accept: application/json
4 authorization: Bearer <JWT>
5 content-length: 2000
6 content-type: application/json
7 content-encoding: gzip
8
9 {
10   user_id: <int>,
11   comment: <object>
12 }
```

The server responds with status code *200 OK* on success.

▼ JavaScript



```
1 status code: 200 OK
```

Alternatively, the server responds with status code *201 created* on success.

▼ JavaScript



```
1 status code: 201 created
```

The server responds with status code *400 bad request* for indicating a failed request due to an invalid request payload from the client.

▼ JavaScript



```
1 status code: 400 bad request
```

The client receives a status code *403 forbidden* when the client has valid credentials but insufficient privileges to act on the resource.

▼ JavaScript



```
1 status code: 403 forbidden
```

Further System Design Learning Resources

Download my system design playbook for free on newsletter signup:



The System Design Newsletter

Download my system design playbook on
newsletter signup for FREE

By Neo Kim 🏆 · Over 235,000 subscribers

amanpunetha08@gmail.com



By subscribing you agree to [Substack's Terms of Use](#), [our Privacy Policy](#) and [our Information collection notice](#)

substack

Live Comment System Database Design

The live comment service is a read-heavy system. In simple words, the predominant usage pattern is the client viewing live comments.



Figure 5: Live comment database schema

The major entities of the relational database are the comments table, the videos table, and the users table. The relationship between the users and the comments tables is **1-to-many**. The relationship between the videos and the comments tables is **1-to-many**. The relationship between the users and videos tables is **1-to-many**.

| Comments table

Column	Description
id	ID to identify the comment
user_id (Foreign key)	publisher of the comment
video_id (Foreign key)	ID of the associated video
content	content of the comment
created_at	timestamp of creation

| Videos table

Column	Description
id	identifier of the video
title	title of the video
user_id (Foreign key)	streamer of the live video
created_at	timestamp of creation

| Users table

Column	Description
id	identifier of the user
name	name of the user
email	email of the user

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

profile_image	URL to fetch the profile image of user.
last_login	last login timestamp of the user
created_at	timestamp of user account creation

| SQL

Structured Query Language (**SQL**) is a domain-specific language for managing data stored in the relational database management system.

| Write a SQL query to fetch the latest ten comments on the live video with "35" as the video ID

✓ SQL



```
1 SELECT *
2 FROM comments
3 WHERE video_id="35"
4 ORDER BY created_at DESC
5 LIMIT 10;
```

| Write a SQL query to insert a new live comment

✓ SQL



```
1 INSERT INTO comments (id, user_id, video_id, content, created_at)
2 VALUES (3, "42", "35", "awesome", "2050-08-22");
```

| Write a SQL query to fetch the total count of comments on each video

✓ SQL



```
1 SELECT COUNT(id), video_id
2 FROM comments
3 GROUP BY video_id;
```

| Type of Data Store

The content of live comments contains only textual data and does not include any media files. A very fast and reliable database that not only persistently store data but also access the data quickly is a key feature for building the live comment service. The persistent storage of live comments is needed to retrieve the comments at a later point in time. In addition, the data kept in persistent storage can be used for auditing purposes¹⁰. The relational database offers well-defined structures for comments, users, and videos. The relational database will be an optimal choice when the dataset is small. However, the relational database will be a suboptimal solution for live comment service due to the following scalability limitations⁵:

- the internal data structure adds delay to the data operations
- complex queries are needed to reintegrate data because of data segregation

The creation of database indexes on *video_id* and *created_at* columns will improve the performance of the read operations at the expense of slow write operations. The NoSQL database such as [Apache Cassandra](#) can be used as persistent data storage for live comments due to the following reasons⁵:

- Log-structured merge-tree (**LSM**) based storage engine offers extremely high performance on writes
- schemaless data model reduces the overhead of joining different tables
- optimized natively for time series data

Apache Cassandra is not optimized for read operations due to the nature of the LSM-based storage engine. An in-memory database such as Redis can be used in combination with Apache Cassandra to improve the read performance and make the data storage layer scalable and performant for live comments. The geo-replication enabled Redis [cache](#) with a time to live (**TTL**) of 1 second can be added as a cache

System Design

network edges to improve the latency ¹⁰.

The sets data type in Redis can be used to efficiently store the live comments. The native deduplication logic of sets data type ensures that live comments are stored in memory without having an additional logic to prevent repeated live comments. The [sorted set](#) data type in Redis can be used for data integrity by maintaining the reverse chronological ordering of live comments. The sorted set data type can make use of the timestamp on live comments for sorting the live comments without implementing a custom sorting algorithm. The metadata of the publisher of a live comment can be stored in Redis hash data type for quick retrieval ^{4, 11, 5}.



Figure 6: Live comments; AP in CAP Theorem

The receivers who are geographically located closer to the publisher of the live comment will see the live comment instantly while the receivers who are located on a different continent might see the live comment with a slight delay (lower than 250 ms) to favor availability and partition tolerance in the CAP theorem ¹⁰.

Capacity Planning

The rate of clients viewing the live comments is significantly higher than the rate of clients publishing the live comments. The calculated numbers are approximations. A few helpful tips on [capacity planning](#) during system design are the following:

- 1 million requests/day = 12 requests/second
- round off the numbers for quicker calculations
- write down the units while doing conversions

| Traffic

The live comment service is a read-heavy system. The Daily Active Users (**DAU**) count is 100 million. On average, the total number of daily live videos is 200 million. A live video receives 10 comments on average.

Description	Value
DAU (write)	2 billion
QPS (write)	12 thousand

QPS (read)	1.2 million
------------	-------------

| Storage

The comments on a streamed live video can be archived and kept in a cold store to save storage costs. In addition, comments older than a specific time frame can be removed to minimize storage costs. The storage size of each character on a live comment is assumed to be 1 byte.

Description	Size (bytes)
id	20
user_id	20
video_id	20
content	2000
created_at	10

Description	Calculation	Total
storage for a day	12k comments/sec * 2 KB/comment * 86400 sec/day	2 TB

In total, a live comment is approximately 2 KB in size. The replication factor for storage can be set to three for improved durability and disaster recovery.

| Bandwidth

Ingress is the network traffic that enters the server when live comments are written. **Egress** is the network traffic that exits the servers when live comments are viewed. The network bandwidth is spread out across the globe depending on the location of the clients.

Description	Calculation	Total
Ingress	2 billion comments/day * 2 KB/comment * 10 ⁽⁻⁵⁾ day/sec	40 MB/sec
Egress	200 billion comments/day * 2 KB/comment * 10 ⁽⁻⁵⁾ day/sec	4 GB/sec

| Memory

The in-memory subscription store keeps the client viewership associations.

Description	Calculation	Total
Subscription store	100 million users/day * 16 bytes/user	1.6 GB/day

Further System Design Learning Resources

Download my system design playbook for free on newsletter signup:

The System Design Newsletter

Download my system design playbook on newsletter signup for FREE

By Neo Kim 🏆 · Over 235,000 subscribers

amanpunetha08@gmail.com



By subscribing you agree to [Substack's Terms of Use](#), [our Privacy Policy](#) and [our Information collection notice](#)

substack

Live Commenting High-Level Design

| Write Globally and Read Locally

Figure 7: Write globally and read locally through asynchronous database replication

The database can be configured to asynchronously replicate the data across the database servers in data centers located on distinct continents. In layman's terms, the data is always fetched by the server from the database servers in the local data center while the data updates are asynchronously written to database servers in data centers across the globe. This technique is known as **writing globally and reading locally**. For instance, when a client located in the US publishes a live comment on a Facebook live video, the clients located in Europe will not see the live comment instantaneously due to the asynchronous nature of data replication. The following are the drawbacks of writing globally and reading locally approach for implementing live comment service ¹²:

- significant bandwidth usage on data replication
- live comments will not be real-time due to asynchronous replication
- poor overall latency

In summary, do not use the writing globally and reading locally approach for implementing live comments.



Figure 8: Write locally and read globally through timestamp query (pull-based model)

The data is always written to the database server in the local data center while the data is fetched by querying the servers in data centers across the globe as shown in Figure 8. This technique is known as the **pull-based model of writing locally and reading globally**. The timestamp on the live comments can be used to check if there are newer comments published since the last query execution. The server in the local data center will consolidate all the published live comments and return the response to the client. The bandwidth usage is relatively lower because the data is not replicated globally. However, the pull-based model will result in degraded latency because the server must query all the data centers across the globe on each read operation ¹². In summary, do not use the pull-based model of writing locally and reading globally approach for implementing live comments.



Figure 9: Write locally and read globally through broadcasting (push-based model)

The data is always written to the distributed database server in the local data center. In Figure 9, when a client publishes a live comment on a Facebook live video, the write operation is broadcast by the server to multiple data centers across the globe. This technique is known as the **push-based model of writing locally and reading globally**. The push-based model of writing locally and reading globally significantly reduces the usage of expensive bandwidth and improves the latency resulting in real-time live comments ¹². In summary, use the push-based model of writing locally and reading globally for implementing live comments.

| Prototyping a Live Comment Service



Figure 10: Prototyping a Live comment service

The clients watching a live video are known as **receivers** and the clients publishing live comments on a live video are known as **publishers**. The server holding the SSE connections to the receivers is known as the **gateway server**. A dedicated disk-based **subscription store** can be provisioned to persist the lifecycle of the client connections. The **dispatcher** is an abstraction layer for publishing data objects such as live comments or [Facebook likes](#) by the client. The following operations are performed when a live comment is published on a live video ¹³:

1. receiver subscribes to a live video on the gateway server
2. gateway server persists the metadata of the client connection in the subscription store
3. the publisher publishes a live comment on a live video
4. dispatcher queries the subscription store for all viewership associations on the live video
5. dispatcher publishes the live comment to the gateway server
6. gateway server delivers the live comment to all subscribed receivers

The gateway server maintains the SSE connections with the clients. The subscription store should be replicated at least thrice for reliability and durability ¹³. The subscription store will become a bottleneck to scaling the live comment service because every read and write operation on live comments must query the subscription store. On top of that, the client connections to a specific live video are short-lived because clients scroll through the Facebook newsfeed. As a result, the subscription store must be updated frequently. In summary, the current prototype will not meet the requirements of a scalable and reliable live comment service.

| Distribution of Live Comments



Figure 11: Distribution of live comments through SSE

The receiver can periodically request (pull-based) the server to check whether new live comments were published. The periodic polling of the server will result in empty responses consuming unnecessary bandwidth. Moreover, the polling interval for live comments should be lower than 250 ms for a real-time experience. The increased polling frequency will highly likely overload the servers. The push-based approach using SSE persistent connections is optimal for the real-time delivery of live comments to the receivers ¹².



Figure 12: In-memory subscription store on gateway server keeping viewership associations

The SSE connections to the client are non-blocking fire-and-forget for improved performance ⁷. An in-memory subscription store using Redis can be provisioned

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

efficiently store the ephemeral viewership associations. The subscription store is queried to identify the receivers who should receive the live comments on a specific live video ¹².



Figure 13: Live comment delivery through the real-time platform

An abstract and scalable real-time platform can be built for real-time experiences such as displaying [real-time presence](#), push notifications, [Facebook likes](#), [Facebook reactions](#), typing indicators, and live comments. The real-time platform can be reused across multiple platforms such as Facebook, Messenger, Instagram, Twitch, TikTok, LinkedIn, or YouTube ¹³. A dynamically configurable plugin system architecture should be implemented to create an abstract real-time platform. Plugins are embedded executable code for additional logic that gets invoked on distinct events such as the publishing of a live comment ³. The principle behind building a scalable distributed system is to start small and iteratively add simple layers to the architecture ⁷. The general guidelines for horizontally scaling a service are the following ¹⁴:

- keep the service stateless
- partition the service
- replicate the service

| Live Commenting With Pub-Sub Server



Figure 14: Pub-sub server for live comments

Publish-Subscribe (pub-sub) pattern allows services running on distinct technologies to communicate with each other ¹¹. The pub-sub technique using the message bus enables a producer to send live comments (**messages**) to multiple consumers. The services can communicate with each other instantly without having preset intervals for polling data from relevant data sources in a reactive architecture ⁵.

| Using Apache Kafka as the Pub-Sub Server



Figure 15: Live comments; Apache Kafka as the pub-sub server

Apache Kafka is configured as the pub-sub server to decouple the producers and consumers. The live comment service can be reliably scaled through the separation of

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

Kafka streaming protocol enables sharding the data before [streaming](#) the data to consumers ⁶, ⁷. The system design of live comments is challenging for the following reasons:

- clients will continuously scroll through the Facebook [newsfeed](#)
- live videos visible on the viewport of the client will change frequently

There will be multiple receivers (**consumers**) watching a live video. The live comments published on the live video should be consumed by all the receivers. When the client scrolls through the Facebook newsfeed or navigates outside the comments panel, the client is not supposed to see the live comments anymore. The consumer should dynamically unsubscribe from the specific live video (**Kafka topic**) to stop seeing the live comments on the specific live video. However, unsubscribing from a Kafka topic is an expensive operation ¹⁵, ¹⁶. In addition, tracking the gateway servers consuming specific live videos is non-trivial resulting in oversubscription to all Kafka topics. The limitations of using Apache Kafka for building the live comment service can be summarized as the following ⁷:

- degraded latency because consumers use a pull-based model
- limited scalability because each consumer should subscribe to all the Kafka topics and consume all the messages
- operational complexity of Apache Kafka is relatively high

The Kafka topics can be partitioned by the stream IDs at the expense of unbalanced partitions. Consumers can subscribe to specific partitions using [consistent hashing](#) ³. Nonetheless, the consumers will not know in advance the stream IDs that the clients will be interested in. In a typical scenario, a server will have a diverse set of clients interested in almost all the live videos resulting in subscriptions to all the live videos. [Apache Pulsar](#) also runs into a similar set of challenges as Apache Kafka for the use case of live comment service ⁷. In summary, do not use Apache Kafka as the pub-sub server for implementing live comments.

| Using Redis as the Pub-Sub Server



Figure 16: Live comments; Redis cluster as the pub-sub server

The clients must use a protocol named REdis Serialization Protocol (**RESP**) for communicating with the Redis server. Redis can be used as the [pub-sub](#) server to transmit live comments (**messages**) between nodes ⁴. The Redis cluster can be used

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

specific node resulting in degraded performance. The **consistent hashing** algorithm is used for load-balancing client subscriptions to live videos between a set of independent Redis nodes for improved scalability. **Consistent hashing** significantly reduces the movement of live connections between nodes on a node failure or node addition. In layman's terms, the incoming messages are published to all Redis nodes but the server only listens on a small set of nodes for a specific subscription (live video) for improved scalability. The Redis replicas should be provisioned on every node to perform automatic failover when a node failure occurs ³.

A TCP connection is maintained between the producer to Redis and Redis to the consumer for delivering messages. As an unconventional workaround, the server could terminate the TCP connection between the consumer node and the pub-sub server for unsubscribing the consumer from a topic (live video) when the client scrolls through the Facebook newsfeed. The drawbacks of using the Redis pub-sub server for live comment service are the following:

- no guaranteed at least one-time message delivery
- reliability of message delivery depends on the TCP connection
- messages might get lost due to a lack of message persistence

In conclusion, do not use the Redis pub-sub server solution for implementing live comments.

| Using Redis Streams as the Pub-Sub Server

The Redis stream is a data structure that can be used like an append-only log for improving the responsiveness of the system. In simple words, Redis streams are functionally very equivalent to Apache Kafka. The Redis streams as the pub-sub server offer the following benefits ^{17, 18, 5, 19, 20, 21}:

- guaranteed at least one-time delivery of messages
- reliable messages through persistence with Append Only File (**AOF**) or Redis Database (**RDB**) persistency methods
- more tolerant to network partitions
- improved latency due to efficient in-memory storage
- relatively trivial to provision and operate
- messages can be consumed either in blocking or non-blocking methods

The Redis Sentinel or Redis Active-Active configuration provides high availability for the cluster at the expense of increased operational complexity. The Redis Sentinel is a distributed system that monitors and provides failover policies to Redis instances. Redis streams could dynamically unsubscribe consumers from a specific topic (live video). However, there is a risk of data loss in Redis streams because the data is only periodically written to disk. The memory limitation could also be a bottleneck to building a scalable live commenting service ^{19, 21}. In conclusion, do not use Redis streams as the pub-sub server solution for implementing live comments.

| An Abstract Real-Time Platform



Figure 17: Live comment; High-level design

An in-memory **subscription store** can be provisioned using Redis on the same machine where the gateway server runs. The in-memory subscription store will hold the mapping between live videos and receivers (client connections). The gateway server can perform throttling and include additional product logic. A dedicated disk-based **endpoint store** can be used to identify the gateway servers interested in a specific live video. The endpoint store will contain a mapping between the live videos and a set of gateway servers. The following operations are performed when a live comment is published on a live video ¹³:

1. receiver subscribes to a live video on the gateway server
2. the gateway server updates the subscription store with viewership association and informs the endpoint store that the gateway server is interested in the specific live video
3. the publisher writes a live comment on the specific live video
4. dispatcher queries one of the replicas of the endpoint store to fetch the set of subscribed gateway servers
5. the dispatcher forwards the live comment to the subscribed gateway servers
6. the gateway server checks the local in-memory subscription store and fans out the live comment to subscribed receivers

The following operations can be executed for removing the expired records from the in-memory subscription store on the gateway server:

- the gateway server can send periodic heartbeat signals to the receivers to check if they are still connected
- Redis TTL keys can expire the subscription record when the heartbeat signals show a failure
- client-side JavaScript logic can trigger an event when the client scrolls away from the live video

The endpoint store should remove the live video only when the set of gateway servers subscribed to the live video is empty. Redis set data type allows checking the cardinality of the set in constant time complexity. The ending of a live video can trigger the expiry of the records in the disk-based endpoint store. In summary, the current architecture can satisfy the requirements of a scalable and reliable live comment service.

Further System Design Learning Resources



The System Design Newsletter

Download my system design playbook on newsletter signup for FREE

By Neo Kim 🏆 · Over 235,000 subscribers

amanpunetha08@gmail.com



By subscribing you agree to [Substack's Terms of Use](#), our [Privacy Policy](#) and our [Information collection notice](#)

substack

Live Comment System Design Deep Dive

Live experiences are underpinned by real-time, event-driven APIs for meeting the requirements of clients. A real-time platform is predicated upon low latency, data integrity, fault tolerance, availability, and scalability ⁴.

| How Does the Gateway Server Manage Client Connections?

An **actor** is an extremely lightweight object that can receive messages and take actions to handle the messages. The actor is decoupled from the source of the message. The actor is only responsible for recognizing the type of message received and performing the required action. A thread will be assigned to an actor when a message must be processed. The thread is released once the message is processed and the thread is assigned to the next actor. The total count of threads will be proportional to the count of CPU cores. A relatively small count of threads can handle a significant number of concurrent actors because a thread is assigned to an actor only during the execution time ^{7, 8, 22}.



Figure 18: Gateway server; Client connection management using the actor model

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

published to the client connection. The actor-style programming can be implemented with actor frameworks such as [Akka](#) (Java, Scala), [Pykka](#) (Python), or [Cloudi](#) (Erlang) for building highly concurrent, distributed, and resilient message-driven applications ^{7, 8, 22}.



Figure 19: Gateway server; Broadcasting live comments using the actor model

The (child) actors are managed by the supervisor actor. The following operations are executed when a live comment is published by the dispatcher ⁷:

1. the dispatcher publishes a live comment to the supervisor actor in the gateway server over the HTTP
2. the supervisor actor broadcasts the live comment to all the child actors
3. child actors use the connection handle to forward the live comment to the receivers

The client (receiver) will render the received live comment on the web browser.

| How to Handle Live Comments on Multiple Live Videos?

There might be multiple clients watching distinct live videos connected to the same gateway server. The actors on the gateway server should not publish the live comment on a specific live video to all the connected clients because some of the clients might be watching a different live video. In Figure 20, clients connected to the same gateway server are watching distinct live videos with live video IDs *blue* and *red*. When the client publishes a live comment on the live video with *blue* as the ID, only the clients watching the live video with *blue* as the ID should be able to view the published live comment.



Figure 20: Gateway server; Publishing live comments on multiple live videos using a subscription

An in-memory subscription store using Redis can be provisioned to run on the same machine where the gateway server runs. The client starting to watch a live video subscribes to the specific live video on the gateway server over HTTP. The **subscription store** will keep an in-memory mapping between a live video and the set of receivers. An in-memory subscription store is used by the gateway server for storing viewership associativity for the following reasons ⁷:

- subscription data is local to clients connected to the gateway server
- the client connections are strongly tied to the lifecycle of the gateway server

In layman's terms, an in-memory subscription store is used to identify the live video watched by each client connected to the gateway server. The following operations are executed when a client publishes a live comment on the live video with *blue* as the ID ⁷:

1. the dispatcher publishes a live comment to the supervisor actor in the gateway server over the HTTP
2. the supervisor actor queries the local in-memory subscription store to identify the clients subscribed to the live video with *blue* as the ID
3. the supervisor actor broadcasts the live comment to all the subscribed child actors
4. child actors use the connection handle to forward the live comment to the receivers

The local in-memory subscription store allows the efficient publishing of live comments on multiple live videos.

| How to Support Massive Concurrent Clients on Multiple Live Videos?



Figure 21: Dispatcher broadcasting live comments between gateway servers

The gateway server should be partitioned by provisioning multiple instances of gateway servers to handle massive concurrent clients as shown in Figure 21. An additional service known as the **dispatcher** is provisioned to broadcast live comments between multiple gateway servers⁷. The clients connected to a gateway server might be watching only a subset of live videos. It is inefficient for the dispatcher to broadcast the live comments on all live videos to all gateway servers because some gateway servers might be subscribed to only a subset of live videos.



Figure 22: Dispatcher broadcasting live comments to all gateway servers

For example, in Figure 22, clients connected to gateway server 1 are only watching the live video with *blue* as the video ID. When the dispatcher publishes live comments on

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

resulting in wasted computing resources. The dispatcher should not publish live comments on all live videos to all the gateway servers for improved performance⁷.



Figure 23: Gateway servers subscribing to an in-memory endpoint store in the dispatcher

An in-memory endpoint store using Redis can be provisioned to run locally on the same machine as the dispatcher. The client starting to watch a live video subscribes on the gateway server over HTTP. The subscription store in the gateway server will keep the mapping between the specific live video and the set of receivers. The gateway server will subscribe to the specific live video on the dispatcher over HTTP. An in-memory **endpoint store** on the dispatcher will keep the mapping between a live video and the set of subscribed gateway servers. An in-memory endpoint store is used by the dispatcher for keeping the subscription data of the gateway servers due to the following reasons⁷:

- efficiently dispatch live comments across multiple gateway servers
- identify the gateway servers subscribed to a particular live video

In layman's terms, an in-memory endpoint store in the dispatcher is used to identify the live video subscription by the gateway servers for improved performance.



Figure 24: Dispatcher publishing live comments to the subscribed gateway servers

The following operations are executed when the client publishes a live comment on the live video with *red* as the ID ⁷:

1. the dispatcher queries the local in-memory endpoint store to identify the subscribed gateway servers on the live video with *red* as the ID
2. the dispatcher publishes the live comment to the subscribed gateway servers over HTTP
3. the gateway server queries the local in-memory subscription store to identify the clients subscribed to the live video with *red* as the ID
4. the gateway server broadcasts the live comment to all the subscribed clients through SSE

The major drawback of the current architecture is that the dispatcher becomes the bottleneck for scalability. The performance of the live comment will degrade on peak load because there is only a single instance of dispatcher deployed.

| Scaling Live Comments to Handle Peak Load

When an extremely high rate of live comments is published per second, the single instance of dispatcher will degrade the overall performance of the live comment service. The dispatcher can be replicated for horizontal scaling. The dispatcher should be made stateless for replicating the dispatcher. The dispatcher can become stateless by moving out gateway subscription state to an external store.

A disk-based key-value store known as the **endpoint store** can persist the subscription associations between a live video and the set of gateway servers. The endpoint store must be replicated for scalability and high availability. The endpoint store should only persist the gateway server subscription local to the data center. DynamoDB, Redis, or MongoDB can be used to provision the endpoint store. In addition, the endpoint store provides durability in case the dispatcher fails. Any dispatcher will be able to identify the set of subscribed gateway servers by querying the external endpoint store ⁷.



Figure 25: Scaling the dispatcher using a key-value endpoint store

The dispatcher communicates with gateway servers over HTTP. Any dispatcher instance will be able to publish live comments to any gateway server. The following operations are executed when the client publishes a live comment on a live video with *red* as the ID ⁷:

1. the dispatcher independently queries the external endpoint store to identify the set of subscribed gateway servers on the live video with *red* as the ID
2. the dispatcher publishes the live comment to the set of subscribed gateway servers over HTTP
3. the gateway server queries the local in-memory subscription store to identify the clients subscribed to the live video with *red* as the ID
4. the gateway server broadcasts the live comment to all the subscribed clients through SSE

The endpoint store doesn't need to be partitioned due to the small dataset.

| What Is the Subscribe Workflow and Publish Workflow for Live Comments?



Figure 26: Live comment; Subscribe workflow when the client starts watching a live video

The following operations are executed for the subscription when the client starts watching a live video ⁷:

1. the client subscribes to the gateway server over HTTP
2. the gateway stores the viewership associations on the in-memory subscription store
3. the gateway server makes a subscription request on the endpoint store by creating an entry on the key-value store



Figure 27: Live comment; Publish workflow when the client publishes a live comment

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

1. the load balancer distributes the live comment published by the client to any random dispatcher over HTTP
2. the dispatcher queries the endpoint store for identifying the set of gateway servers subscribed to the particular live video
3. the dispatcher forwards the live comment to the set of subscribed gateway servers over the HTTP
4. the gateway servers query the local in-memory subscription store to identify the subscribed clients for the particular live video
5. the gateway servers broadcast the live comments to the subscribed clients over SSE

| How to Deploy the Live Comment Service Across Multiple Data Centers?

The live comment service should be deployed across multiple data centers (**DC**) for scalability and performance. There might be a scenario when no clients are watching a particular live video on some data centers. The dispatcher can query the local endpoint store to identify if there are any gateway servers subscribed to the particular live video in the local data center. The endpoint store keeps only gateway servers subscription local to the data center ⁷.

For instance, in Figure 28, no clients are watching the video with *red* as the video ID on data centers one and three. When a client publishes a live comment on the live video with *red* as the video ID, only the dispatcher on data center two should broadcast the live comment to the subscribed gateway servers. The dispatcher must broadcast the live comment to the dispatchers in peer data centers for ensuring that the clients subscribed to the particular live video on peer data centers receives the live comment in real-time.



Figure 28: Publishing the live comment across data centers through broadcasting

The following operations are executed when the client publishes a live comment on the live video ⁷:

1. **GeoDNS** routes the live comment to a dispatcher on the data center closest to the client

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

3. the dispatcher queries the local endpoint store to check if there are any subscribed gateway servers on the particular live video
4. the subscribed gateway server queries the local in-memory subscription store to identify the subscribed clients
5. the gateway server fans out the live comment to the subscribed clients over SSE

The current architecture is scalable and will satisfy the requirements of live comment system design. The alternative approaches for implementing cross-data center live comment service are the following:

- cross-data center subscription
- endpoint store configured for the entire region

When the client starts watching a live video, the gateway server could perform a cross-data center subscription. However, the client connections are ephemeral because the client might scroll through the Facebook newsfeed, and the gateway servers subscription should be frequently updated across cross-data centers causing poor performance and degraded latency.

Alternatively, the endpoint store can be configured globally for keeping gateway servers subscriptions across multiple data centers covering an entire region. When a key-value store such as DynamoDB is used for the endpoint store, the [quorum](#) should be tuned to higher [consistency](#) to ensure that all subscribed gateway servers will receive live comments. However, the global endpoint store will result in some clients not receiving the live comments due to eventual consistency and also cause poor latency.

| How to Support the Typing Indicators on Live Video?

The JavaScript logic using [event listeners](#) on the client can trigger events when the user starts typing a live comment. The real-time platform can broadcast the typing event to the subscribed clients. The web browser on the subscribed clients can display the typing indicator.

| How to Display the Total Count of Comments on Each Live Video?

[HyperLogLog](#) can be used to show the approximate count of total comments on each live video in a space-efficient manner. [Redis](#) offers out-of-the-box support for HyperLogLog probabilistic data structure.

Download my system design playbook for free on newsletter signup:



[The System Design Newsletter](#)

Download my system design playbook on newsletter signup for FREE

By Neo Kim 🏆 · Over 235,000 subscribers



By subscribing you agree to [Substack's Terms of Use](#), [our Privacy Policy](#) and [our Information collection notice](#)

substack

| Scalability



Figure 29: Performance of live comment distribution

The dispatcher provisioned on a modern server can handle up to 5000 requests per second. A modern gateway server implemented with the actor model can hold up to 100 thousand concurrent SSE client connections. The multiplication factor introduced by the combination of the dispatcher and gateway servers makes the live commenting service extremely scalable. Another multiplication factor is within the gateway server using the [actor model](#). The actor model allows the reuse of a pool of threads for increased throughput. The live comment service can be horizontally scaled by provisioning new servers ^{7, 22, 2}.



Figure 30: Load balancing client connection to the gateway servers

Load balancers should be introduced between different layers of the system for scalability and improved fault tolerance. The services should utilize the full capacity of

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

complex architecture, and increased infrastructure and maintenance costs ⁴.

| Latency

The live comment service is deployed on multiple data centers to keep the clients closer to the servers to improve the latency. [Apache Samza](#) can be used to measure the end-to-end latency of the live comment microservice system ²⁴. The latency of the live comment service is quite low due to the following reasons ^{4, 7, 23}:

- there is only one key value lookup from the (disk-based) endpoint store
- there is only one in-memory lookup from the subscription store
- the network hops are very few

Additionally, Redis can be configured in the cache-aside pattern for improving the performance of endpoint store lookups because frequent disk I/O might become a bottleneck. The TTL on the Redis cache must be set to a reasonably short timeframe for cache invalidation.

| Concurrency

The gateway servers can be tuned to improve the count of concurrent connections supported by the server. The following operations could be performed to improve the performance of the gateway servers ^{8, 23}:

- increase the thread count limit by decreasing the stack size per thread
- increase the thread count limit by decreasing the memory allocated to the heap
- increase the limit on the number of open connections between the load balancer and the server
- increase the per-process file descriptor limit
- modify the kernel parameter to increase the size of the backlog of TCP connections accepted by the server

The [readers-writer lock](#) can be used for updating the set of gateway servers subscribed to a particular live video on the endpoint store. The client subscribing to a gateway server is handled by the in-memory subscription store running on Redis. The concurrency problems are automatically solved because the operations on Redis are atomic and single-threaded ⁵.

| High Availability



Figure 31: The traffic pattern for live comments on a live video

The traffic pattern for live comments on extremely popular live videos is usually very steep from the initial phase until the end of the live video and the traffic subsequently drops to the floor. In layman's terms, the traffic pattern for live comments on a live video is spiky. When the live video becomes extremely popular, the number of concurrent clients subscribed for viewing the live comments increases causing the [thundering herd problem](#) ². The thundering herd problem from live comments on the live video can be resolved by the following operations ^{13, 2}:

- load balancer redirects the clients to gateway servers with free capacity
- include jitter on client reconnection logic
- implement backpressure and exponential backoff on services
- concurrent clients can perform request coalescing for fetching older comments
- predicting the load before hitting the service limits

The reconnection storms on client SSE connections to the gateway servers on deployment and scale-down events can be prevented by configuring the deployment to keep the existing servers for a few hours after the termination to allow the majority of the existing client SSE connections to close naturally. The tradeoff with the approach is relatively slower deployments ³. The services should operate in hot-hot mode with failover for high availability. The read-write operations can be handled by both instances for scalability ¹⁰.

| Fault Tolerance

The live comment service should be load tested for identifying failures in capacity estimation ². Rate limiters can be used to reduce the load from the service on service degradation or traffic overloading ³. A high level of observability through monitoring and alerting should be configured on all the services ^{3, 23}. The live comment service must be deployed across multiple data centers in the same region and across multiple regions across the globe for improved fault tolerance. The deployment of the live comment service on multiple data centers across the globe comes with the challenge of increased engineering, DevOps efforts, and infrastructure costs ⁴.

| Development and Deployment

Infrastructure as Code can be used to provision servers on bare metal. Continuous integration and continuous delivery/continuous deployment (CI/CD) should be set up

| Durability

The live comments should be persisted in a NoSQL database such as Apache Cassandra for durability. The compaction process in Apache Cassandra for removing the tombstones can become a potential performance bottleneck over time. The database can be configured in different clusters for different profiles of data usage to overcome the limitation. For example, different replication rules can be set for different regulatory requirements (HIPAA, GDPR). Database snapshots must be taken at periodic intervals for data recovery. The data can be replicated on multiple hyper scalers within the same region for durability ¹⁰.



Figure 32: Storing live comments in the NoSQL database

The following operations are executed when the client publishes a live comment on the live video:

1. the load balancer distributes the live comment published by the client to any random dispatcher over HTTP
2. the dispatcher writes the live comment on the comment store (NoSQL) for persistent storage
3. the dispatcher queries the endpoint store for identifying the set of gateway servers subscribed to the particular live video
4. the dispatcher forwards the live comment to the set of subscribed gateway servers over the HTTP
5. the gateway servers query the local in-memory subscription store to identify the subscribed clients for the particular live video
6. the gateway servers broadcast the live comments to the subscribed clients over SSE

| Operational Complexity

Fully managed services hosted on hyper scalers like [AWS](#) can reduce the operational complexity of the live comment service. Alternatively, the dispatcher in the live comment service can be implemented with serverless functions. The benefits of using serverless functions are the following ⁴:

- no infrastructure maintenance

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

- reduced latency

Summary

The live comment is a popular system design interview question. The real-time platform built for publishing live comments can display Facebook reactions, likes, concurrent viewer counts, [user presence status](#), online polls, or seen receipts.

What to learn next?

Download my system design playbook for free on newsletter signup:



The System Design Newsletter

Download my system design playbook on newsletter signup for FREE

By Neo Kim 🏆 · Over 235,000 subscribers



By subscribing you agree to [Substack's Terms of Use](#), [our Privacy Policy](#) and [our Information collection notice](#)

substack

License

CC BY-NC-ND 4.0: This license allows reusers to copy and distribute the content in this article in any medium or format in unadapted form only, for noncommercial purposes, and only so long as attribution is given to the creator. The original article must be backlinked.

References

1. [What is live streaming?](#), [cloudflare.com](#) ↩
2. Todd Hoff, [How Facebook Live Streams To 800,000 Simultaneous Viewers](#) (2016), [highscalability.com](#) ↩
3. Dima Zabello, Kyle Maxwell, and Saurabh Sharma, [Reddit's new real-time service](#), [reddit.com](#) ↩
4. Matthew O'Riordan, [Using Serverless WebSockets to Enable Real-Time Messaging](#) (2022), [infoq.com](#) ↩
5. Fernando Doglio, [Using Redis for Chat and Messaging](#) (2022), [memurai.com](#) ↩
6. Matthew O'Riordan, [The Challenges of Building a Reliable Real-Time Event-Driven Ecosystem](#) (2020), [infoq.com](#) ↩
7. Akhilesh Gupta, [Streaming a Million Likes/Second: Real-Time Interactions on Live Video](#), [infoq.com](#) ↩
8. Akhilesh Gupta, [Instant Messaging at LinkedIn: Scaling to Hundreds of Thousands of Persistent Connections on One Machine](#) (2016), [engineering.linkedin.com](#) ↩
9. [Using server-sent events](#), [mozilla.org](#) ↩
10. Todd Greene, [Efficiently Operating a Mass Real-time Data Infrastructure](#) (2020), [AWS Events](#) ↩
11. Ajeet Raina, [How to build a Chat application using Redis](#), [developer.redis.com](#) ↩

System Design

- 14. [Web Scalability for Startup Engineers](#) by Artur Ejsmont (2015) ↩
- 15. [Offsets retention minutes in Kafka](#), [kafka.apache.org](#) ↩
- 16. [Manage consumer groups in Kafka](#), [docs.confluent.io](#) ↩
- 17. [Redis Streams](#), [redis.io](#) ↩
- 18. [Redis Streams Explained](#) (2021), [youtube.com](#) ↩
- 19. Fernando Doglio, [Apache Kafka versus Redis Streams](#) (2021), [memurai.com](#) ↩
- 20. [Redis persistence](#), [redis.io](#) ↩
- 21. Antirez, [Streams: a new general purpose data structure in Redis](#), [antirez.com](#) ↩
- 22. Akhilesh Gupta, [Now You See Me, Now You Don't: LinkedIn's Real-Time Presence Platform](#) (2018), [engineering.linkedin.com](#) ↩
- 23. Todd Hoff, [Justin.Tv's Live Video Broadcasting Architecture](#) (2010), [highscalability.com](#) ↩
- 24. Max Wolffe, [Samza Aeon: Latency Insights for Asynchronous One-Way Flows](#) (2018), [engineering.linkedin.com](#) ↩

Updated on 2023-05-18

CC BY-NC 4.0



[Back](#) | [Home](#)



Sponsored

Play War Thunder now for free

War Thunder

Play Now

World's Tallest Temple for Krishna in Progress

Vrindavan Chandrodaya Mandir

Click Here

Join new Free to Play WWII MMO War Thunder

War Thunder

Play Now

Karnataka: Support World's Tallest Religious Monument

Vrindavan Chandrodaya Mandir

Click Here

India's Biggest Startup Club now in Bengaluru

Global Startups Club

Register

Bee Keepers of India Selling Pure Honey online to Indian consumers via Ecommerce Website

Scrolling Bee India

Read More

by Taboola



System Design

[Join Newsletter](#)[Archive](#)[About](#)

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS



2

Share

Best

Newest

Oldest

**Navin Pai**

3 years ago

Isn't websockets push based model?

3

0

Reply

**Namrata Vichare**

3 years ago

This is one of the best system design blogs I have read. Thank you making such a wonderful and detailed article. I hope to see more of your blogs later. ❤️

1

0

Reply

**NK**

Author

Namrata Vichare

3 years ago

hey Namrata, thanks for the feedback. I'm actively posting on my weekly newsletter.

0

0

Reply

**Chikeluba**

3 years ago

This is very informative. Thank you so much.

1

0

Reply

**NK**

Author

Chikeluba

3 years ago

you are welcome!

0

0

Reply

**Jack Jackfruit**

3 years ago

Thanks for another great article!

1

0

Reply

**yhyhxiaozi**

a year ago

If DAU (write) 2 billion isn't QPS (write) 24k/s? Given 1 million DAU ~= 12QPS

0

0

Reply

**Trueblue711**

2 years ago edited

Can you share where you got your usage statistics from? The numbers seem a bit high:

200M live videos created daily?

That's 10% of Instagram's MAU. Perhaps watching livestreams, but creating them this seems extremely high.

80M peak concurrent users watching the same live video?

I tried finding statistics but none are reputable or come close to these numbers. One article claims the most watched livestream on Instagram had only 3M concurrent viewers

